Backend architecture & connections

Hospital server deployment • React + Laravel PHP + MySQL

ONE HOSPITAL.
ONE SHARED RECORD.

A practical blueprint for how staff workstations, the Laravel API, clinical workflows, and a shared MySQL database connect across the hospital network.

Document control	Value
Prepared for	EasyCare development team
Version / date	1.1 / 4 November 2026
Status	Reviewed proposal; MySQL and Laravel implementation tests pending
Confirmed decisions	React frontend; Laravel PHP backend; MySQL; hospital-hosted data
Implementation status	Frontend exists. backend/ contains a README only; this document is not an implemented backend.

Purpose

Reviewed against the frontend. Sections 22–30 cover gaps, database revisions and validation limits. The attached v1.1 SQL/dictionary defines the proposed physical schema. MySQL runtime and Laravel workflow tests remain required.

01 / Reading guide & decisions

Pages	What the team will find
3–6	Network topology, connection details, Laravel internals and authentication
7–10	Database relationship diagrams: identity, clinical care, pharmacy and HMO
11–14	Core entity dictionary and database integrity rules
15–17	Workflow transactions, state transitions and API contracts
18–20	Deployment, backups, frontend integration and implementation plan
21–22	Review decisions and technical references
23–31	Frontend audit, full workflow coverage, revised relationships and verification limits

Architecture decisions

• Use one modular Laravel application and one shared database. Modules share patient and encounter IDs; they do not maintain separate copies of patient records.

• Serve the compiled React app and API from the same HTTPS origin. Use Laravel Sanctum session authentication for this first-party browser app [S1].

• Keep uploaded files on private server storage; keep file metadata and clinical links in MySQL. Download through authorized Laravel routes.

• Use InnoDB transactions and row locks for shared resources such as beds and medicine stock [S2, S4].

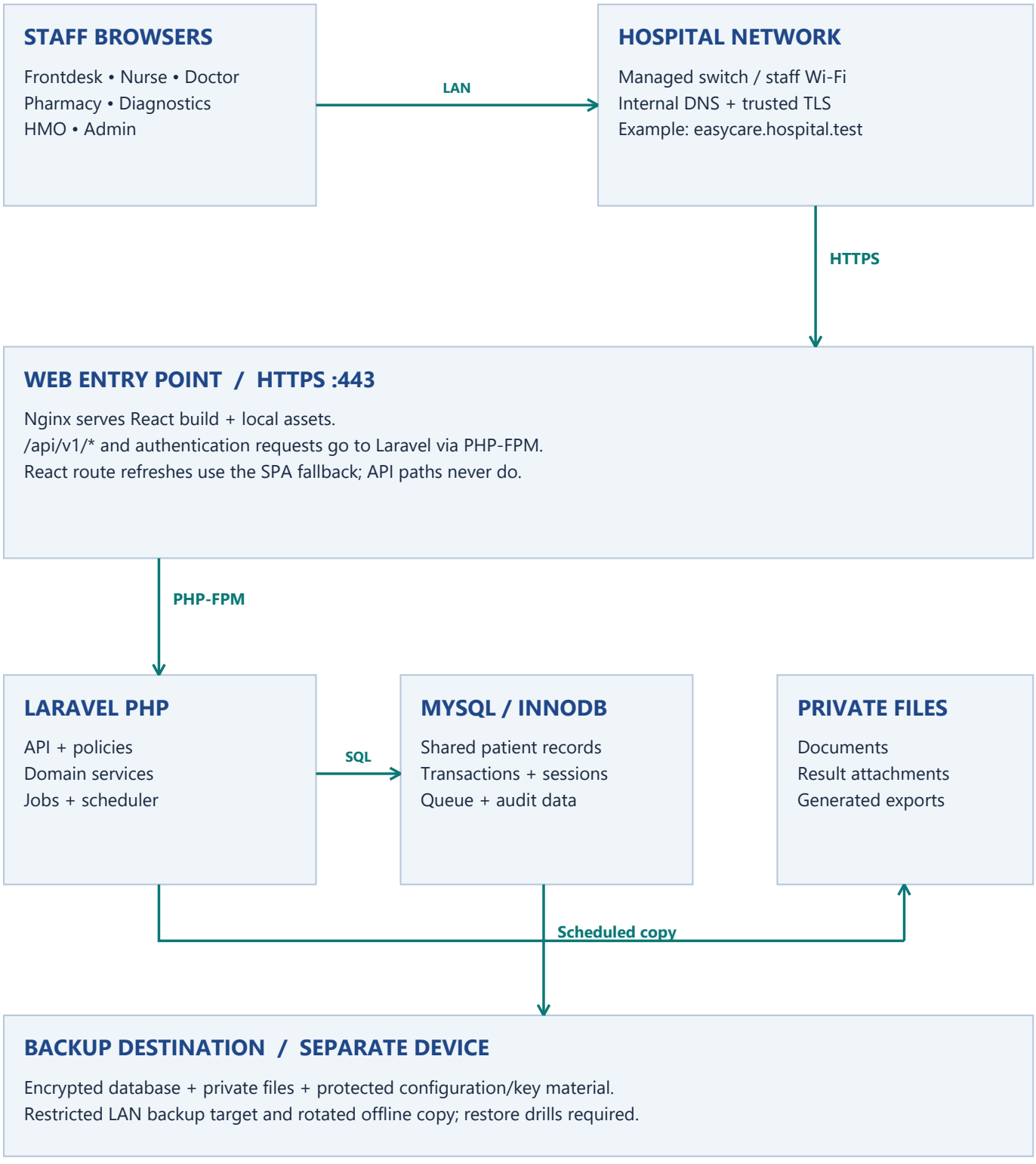
• Run routine hospital work without internet. This requires a working LAN, powered server and locally bundled fonts, scripts and assets.

Scope

Includes the existing staff modules, shared clinical records, inventory, diagnostics and HMO billing. External insurer submission, online payment gateways, multi-hospital tenancy, PACS/DICOM and national health integrations are future interfaces, not assumed capabilities.

02 / Hospital network topology

Logical deployment: one hospital server; multiple staff workstations.



Example hostname only; hospital IT must allocate its real DNS name and certificate.

03 / Connections & trust boundaries

Connection	Protocol / target	Rules
Staff browser → server	HTTPS / TCP 443	Only staff network can reach application. HTTP may redirect to HTTPS.
Nginx → PHP-FPM	Unix socket (Linux baseline)	Local process connection; PHP code is never served as source.
Laravel → MySQL	TCP 3306 on 127.0.0.1	MySQL listens locally in the initial single-server deployment. No browser access.
Laravel → private files	Local filesystem	Dedicated service account permissions; outside public web root.
Queue / scheduler → app	Local PHP processes	Database queue initially; supervisor restarts worker. Scheduler runs each minute.
Server → backup device	Restricted backup transport	Encrypted copy to approved device; destination credentials scoped to backups.
IT administrator → server	Restricted management channel	Allow only designated admin devices. Never expose server tools to general staff.

Configuration contract

Production: APP_URL=https://easycare.hospital.test; React API base=/api/v1; DB_HOST=127.0.0.1; DB_PORT=3306; DB_DATABASE=easycare. Secrets stay in server configuration, outside frontend builds and version control.

Use an application database account with only required runtime privileges. Use a separate controlled migration account. If MySQL later moves to a second server, allow its port only from the app server and enable verified database TLS.

What “local” means

Data is centralized on the hospital server, not in individual browsers. Loss of LAN/server access means the app cannot safely write records. Initial scope has no offline browser write queue or automatic offline conflict resolution.

04 / Inside the Laravel backend

Layer	Responsibility	Example
Routes + middleware	Identify endpoint; validate session, account status, request limits and CSRF where applicable.	POST /api/v1/admissions
Form Requests + policies	Validate input and authorize the action against the actual record.	AdmitPatientRequest; AdmissionPolicy
Thin controllers	Translate HTTP input/output; call one application action.	AdmissionController.store
Domain actions / services	Apply business rules, lock rows, commit atomic changes and write audit events.	AdmitPatient; AssignBed; DispenseMedication
Eloquent + MySQL	Persist models, relationships, keys, constraints and indexed queries.	Patient → encounters → admissions
API Resources	Expose approved fields in consistent JSON; avoid leaking model internals.	PatientResource; AdmissionResource
Jobs / scheduler	Generate large exports, send local reminders and perform maintenance after commit.	GenerateReport; ExpireDownload

Proposed code organization

app/Domain/{Identity,Patients,Clinical,Admissions,Diagnostics,Pharmacy,Billing}; app/Http/{Controllers/Api/V1,Requests,Resources}; app/Policies; app/Jobs; database/{migrations,factories,seeders}; tests/{Feature,Unit}. Keep one deployment and use service calls across modules.

One source of truth

A nurse's discharge and a doctor's admission list must query the same admission record. Reports derive from authoritative tables; they are not separately editable copies of clinical activity.

05 / Login, sessions & permissions

Browser authentication sequence [S1]

1. Staff opens the app over HTTPS and requests /sanctum/csrf-cookie.

2. React posts credentials to /login with cookies and the X-XSRF-TOKEN header. Laravel validates the account, rate limits attempts and rotates the session ID.

3. Browser retains the Secure, HttpOnly session cookie. React calls GET /api/v1/me to obtain identity and allowed capabilities.

4. Every protected request checks auth:sanctum, active status and record-level policy. Logout invalidates the server session and clears browser query data.

Role	Default scope to review with clinical leads
Frontdesk	Register/search patients; book visits; manage demographics. No clinical signing or dispensing.
Doctor	View authorized clinical records; document care; prescribe; order tests; authorize clinical discharge.
Nurse	Record observations; manage ward allocation and discharge execution under hospital policy.
Diagnostics / radiology	See relevant orders; enter results; release only with the required verifier capability.
Pharmacy	View relevant prescriptions; dispense and manage stock according to assigned permissions.
HMO / billing	Manage cover, tariffs, charges and claims; receive only clinically necessary billing context.
Admin	Manage staff and configuration. Clinical access is separately granted and audited.

Permissions use roles + policies [S3]; staff may hold multiple roles. No self-selected admin registration. Bootstrap the first admin locally, then use authorized staff provisioning and local password-reset procedures. Internet email/OTP is optional, not required for daily access.

06 / ERD: identity & patients

Shared identity across all modules; operational roles do not create separate patients.



Parent → child	Cardinality and foreign key
users / roles → user_roles	Many-to-many via two non-null FKs; composite PK prevents duplicate assignments.
roles / permissions → role_permissions	Many-to-many; PK(role_id, permission_id).
patients → encounters / appointments	1 → N each; encounter.appointment_id? UNIQUE links an attended booking.
patients → contacts / flags / documents	1 → N each; documents may optionally link to an encounter.
users → authored records	1 → N through created_by, author_id, ordered_by and other explicitly named FKs.

Legend: PK = primary key; FK = foreign key; ? = nullable. 1 → N means each child has one parent and a parent can have zero or many children. Lines show selected relationships; the table lists the remaining links.

07 / ERD: care & diagnostics

An encounter is a patient visit/care episode; admissions and orders attach to it.

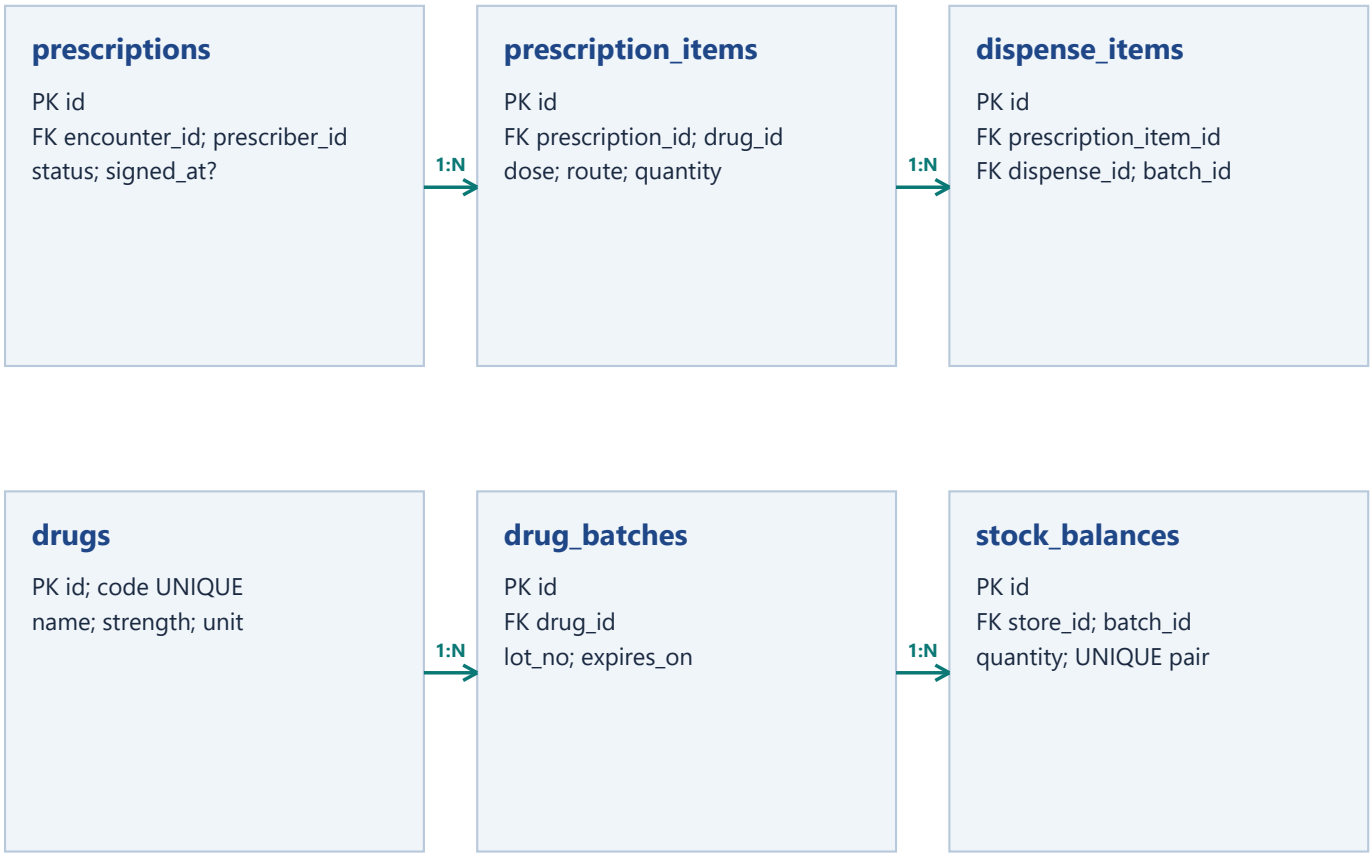


Parent → child	Cardinality and foreign key
encounters → admissions	1 → 0..1; repeat admissions create new encounters.
wards → beds → bed_allocations	1 → N at each step; allocation.admission_id links the admission history.
admissions → current_bed_occupancies	1 → 0..1; bed_id PK and admission_id UNIQUE enforce one active occupant each.
encounters → clinical_entries / vitals	1 → N each. Signed entries have immutable versions/amendments.
orders → items → result_versions	1 → N each. result_versions.item_id FK; UNIQUE(item_id, version).
tests → investigation_items	1 → N; catalog defines result schema, units and validation.

Legend: PK = primary key; FK = foreign key; ? = nullable. 1 → N means each child has one parent and a parent can have zero or many children. Lines show selected relationships; the table lists the remaining links.

08 / ERD: pharmacy & inventory

Prescription intent and physical stock movements are separate, linked records.



Parent → child	Cardinality and foreign key
prescriptions → prescription_items	1 → N; drugs → prescription_items is 1 → N.
dispenses → dispense_items	1 → N. A prescription item can have multiple partial dispenses.
drugs → drug_batches	1 → N; UNIQUE(drug_id, lot_no); batches track expiry.
stores / batches → stock_balances	Many-to-many through balances; UNIQUE(store_id, batch_id).
balances → stock_movements	1 → N; movement.balance_id FK. Ledger is append-only.
purchases → purchase_items	1 → N; FK supplier_id on purchase; item.batch_id FK. Posting creates receipt movements.

Legend: PK = primary key; FK = foreign key; ? = nullable. 1 → N means each child has one parent and a parent can have zero or many children. Lines show selected relationships; the table lists the remaining links.

09 / ERD: HMO, charges & claims

Coverage is time-bound; invoices preserve the price agreed at the time of service.



Parent → child	Cardinality and foreign key
providers → plans → coverages	1 → N at each step; patients → coverages is 1 → N.
plans / services → tariffs	Many-to-many over time; tariff.plan_id + service_id + effective_from UNIQUE.
encounters → invoices → invoice_items	1 → N at each step. Item stores immutable charged price and payer share.
invoices → claims	1 → 0..1 in initial scope; claim_events tracks resubmission/status history.
coverages → preauthorizations	1 → N; authorization also references encounter_id and service_id.
receipts / invoices → allocations	Many-to-many through payment_allocations; deposits can precede invoices. See section 27.

Legend: PK = primary key; FK = foreign key; ? = nullable. 1 → N means each child has one parent and a parent can have zero or many children. Lines show selected relationships; the table lists the remaining links.

10 / Data dictionary: foundations

Selected physical tables. The complete 85-table dictionary is attached to this PDF.

Table	Columns / unique constraints
users	id, username, staff_code, email, password, first_name, last_name, middle_name, is_active, last_login_at, email_verified_at, remember_token, version, department_id*. Unique: username; email; staff_code
patients	id, mrn, first_name, last_name, middle_name, sex, date_of_birth, dob_estimated, email, phone, marital_status, religion, occupation, state_of_origin, address, life_status, deceased_at, access_status, version, category_id*, created_by*, photo_file_id*. Unique: mrn
patient_contacts	id, name, relationship, phone, priority, patient_id*. Unique: patient_id + priority
appointments	id, first_name, last_name, phone, starts_at, ends_at, booking_channel, visit_type, specialist_name, specialist_email, status, patient_id*, doctor_id*, booked_by*
encounters	id, visit_type, care_setting, status, opened_at, closed_at, version, patient_id*, appointment_id*, category_id*, opened_by*. Unique: appointment_id; id + patient_id
documents	id, category, description, patient_id*, encounter_id*, file_id*. Unique: id + encounter_id
files	id, storage_path, original_name, mime_type, byte_size, sha256, scan_status, uploaded_by*. Unique: storage_path
audit_events	id, action, subject_type, subject_id, request_id, occurred_at, safe_summary, actor_id*

* marks foreign-key columns. All tables have primary keys; timestamps omitted here. Full types, nullability, targets, checks and every supporting table are in Database_Dictionary_v1.1.md and schema.mysql.sql. These are proposed definitions, not applied migrations.

11 / Data dictionary: clinical care

Selected physical tables. The complete 85-table dictionary is attached to this PDF.

Table	Columns / unique constraints
clinical_entries	id, data, status, observed_at, signed_at, amendment_reason, version, encounter_id*, template_id*, author_id*, signed_by*, amends_entry_id*
admissions	id, status, admitted_at, authorized_at, discharged_at, discharge_reason, discharge_summary, version, encounter_id*, admitted_by*, authorized_by*, discharged_by*. Unique: encounter_id
bed_allocations	id, assigned_at, released_at, bed_id*, admission_id*, assigned_by*, released_by*. Unique: id + bed_id + admission_id
current_bed_occupancies	bed_id*, admission_id*, allocation_id*. Unique: admission_id; allocation_id
investigation_tests	id, code, name, department, specimen, is_active, service_id*. Unique: code
investigation_schemas	id, version, schema_json, test_id*. Unique: test_id + version; id + test_id
investigation_items	id, priority, status, specimen_ref, collected_at, order_id*, test_id*, encounter_id*. Unique: id + test_id; id + encounter_id
result_versions	id, version, values_json, reference_snapshot, observed_at, status, released_at, amendment_reason, item_id*, test_id*, schema_id*, encounter_id*, entered_by*, verified_by*, attachment_id*, supersedes_id*. Unique: item_id + version

* marks foreign-key columns. All tables have primary keys; timestamps omitted here. Full types, nullability, targets, checks and every supporting table are in Database_Dictionary_v1.1.md and schema.mysql.sql. These are proposed definitions, not applied migrations.

12 / Data dictionary: pharmacy

Selected physical tables. The complete 85-table dictionary is attached to this PDF.

Table	Columns / unique constraints
drugs	id, code, name, category, medication_type, strength, form, base_unit, is_active. Unique: code
stock_balances	id, quantity, version, store_id*, batch_id*. Unique: store_id + batch_id
stock_movements	id, signed_quantity, movement_type, source_key, reason, occurred_at, balance_id*, actor_id*, reversal_of_id*. Unique: source_key
prescription_items	id, dosage, route, frequency, duration, quantity, instructions, status, prescription_id*, drug_id*. Unique: id + prescription_id; id + drug_id
dispense_items	id, quantity, unit_price, dispense_id*, prescription_id*, prescription_item_id*, batch_id*, drug_id*. Unique: id + prescription_item_id
purchase_items	id, units_per_pack, packs_purchased, quantity, unit_cost, selling_price_snapshot, purchase_id*, batch_id*, store_id*
stock_adjustment_items	id, quantity_delta, disposition, refund_amount, adjustment_id*, balance_id*, dispense_item_id*, requisition_item_id*
medication_administrations	id, scheduled_at, administered_at, quantity, dose_unit, status, remark, prescription_item_id*, dispense_item_id*, administered_by*

* marks foreign-key columns. All tables have primary keys; timestamps omitted here. Full types, nullability, targets, checks and every supporting table are in Database_Dictionary_v1.1.md and schema.mysql.sql. These are proposed definitions, not applied migrations.

13 / Data dictionary: finance & system

Selected physical tables. The complete 85-table dictionary is attached to this PDF.

Table	Columns / unique constraints
hmo_providers	id, code, name, payer_type, preauth_required, office_address, phone, alternate_phone, contact_person, email, billing_frequency. Unique: code
patient_coverages	id, member_no, group_number, employer_name, valid_from, valid_until, verification_status, verified_at, patient_id*, plan_id*, verified_by*. Unique: id + patient_id
tariffs	id, effective_from, effective_until, price, currency, plan_id*, service_id*. Unique: plan_id + service_id + effective_from
invoice_items	id, description, quantity, unit_price, discount, net_amount, patient_share, insurer_share, billing_source_key, invoice_id*, service_id*, preauthorization_id*. Unique: billing_source_key
receipts	id, receipt_no, type, amount, currency, method, reference, payer_name, payer_phone, paid_at, description, patient_id*, received_by*. Unique: receipt_no; id + patient_id
payment_allocations	id, amount, receipt_id*, invoice_id*, patient_id*. Unique: receipt_id + invoice_id
claims	id, status, external_ref, submitted_at, invoice_id*. Unique: invoice_id
reminders	id, title, note, due_at, snooze_minutes, recurrence_rule, status, owner_id*, patient_id*

* marks foreign-key columns. All tables have primary keys; timestamps omitted here. Full types, nullability, targets, checks and every supporting table are in Database_Dictionary_v1.1.md and schema.mysql.sql. These are proposed definitions, not applied migrations.

14 / Integrity & simultaneous updates

Rule	Enforcement
Stable relationships	BIGINT UNSIGNED keys; matching FK types; InnoDB; utf8mb4. Index FKs and common filters. Never use names or display MRNs as relational keys.
No orphaned clinical records	Non-null patient/encounter FKs where required. Restrict deletion of referenced clinical/finance data; deactivate catalogs and staff instead.
No double booking a bed	Lock admission and target bed rows; validate no current occupancy; insert current_bed_occupancies inside one transaction. PK bed_id + UNIQUE admission_id are final safeguards.
No negative medicine stock	Lock prescription item and balances in deterministic order; validate remaining prescribed amount, expiry and stock; append movements and update balances atomically.
Prevent silent overwrite	Use version integer / If-Match for editable records. Conditional update increments version; stale version returns 409 with refresh instructions.
Safe retry of important writes	Require Idempotency-Key on registration, admission, dispensing and invoice posting. Same key/hash replays result; same key/different payload returns 409.
Accurate prices and quantities	Use DECIMAL, never floating point for money. Server computes totals and validates non-negative values. Store currency, units and price snapshots.
Signed records remain traceable	Drafts may change with permission. Signed clinical records and released results receive amendments/new versions; never overwrite the original.
Useful indexes	patients(mrn UNIQUE); encounters(patient_id, opened_at); admissions(status, admitted_at); orders(status, requested_at); movements(balance_id, occurred_at); audit(subject_type, subject_id, occurred_at).

Laravel transactions roll back failed database operations [S2]. InnoDB locking reads protect selected rows until commit/rollback [S4]. Multi-step business checks still belong in application services; FKs alone do not enforce clinical consistency.

15 / Workflow: patient to discharge

Step / owner	Backend action and stored relationships
1. Register / Frontdesk	Validate demographics and possible duplicate matches. Allocate server-generated MRN; commit patient + emergency contact + coverage metadata + audit. Return 201 only after commit.
2. Open visit / Frontdesk	Create encounter(patient_id); attach optional appointment and assign a doctor. All departments use this encounter ID.
3. Assess / Nurse + Doctor	Save vital_observations and clinical_entries against encounter_id. Doctor signs entries and may create prescriptions or investigation orders.
4. Admit / Authorized staff	Create admission(encounter_id). Lock chosen bed + admission; append allocation and current occupancy together. Return 409 if the bed was just taken.
5. Provide care / Departments	Nurse records observations, diagnostics releases results, pharmacy dispenses. Lists refresh from the shared API; no department creates a second patient.
6. Authorize / Doctor	Record discharge authorization with actor/time. Financial clearance is a separate configurable requirement, not a substitute for clinical authorization.
7. Complete / Authorized staff	In one transaction: lock admission; validate authorization; set discharge details; release allocation; remove current occupancy; write audit. Close encounter only when required tasks are complete.

State rules

Encounter: open → in_progress → closed (or cancelled before care). Admission: admitted → discharge_authorized → discharged. Bed movement appends allocation history; it does not create a new admission. Re-admission creates a new encounter/admission, preserving the previous episode.

Concurrency example

If two nurses select bed A12, only the first committed allocation succeeds. The second receives a conflict and reloads availability; it must not see a false success screen.

16 / Workflow: orders, stock & claims

Diagnostics: draft → requested → in_progress → released

Doctor creates order + items linked to the encounter. Diagnostics records versioned result values using the test schema, units and reference ranges. An authorized verifier releases a result; only then is it exposed as final to clinical users. Corrections create a new result version with author, reason and verification history.

Pharmacy: signed prescription → partial / full dispense

Pharmacist submits prescription item IDs and requested quantities. Laravel locks prescription items and eligible batch balances, allocates unexpired stock using first-expiry-first-out, and checks cumulative dispensing. One transaction creates the dispense, items, negative ledger movements and balance updates. Any failure rolls back all database changes.

Inventory: draft receipt / transfer → posted

Posting a purchase creates positive movements once. A transfer locks both stores' relevant balances and posts matched outgoing/incoming movements atomically. Quantity changes require reasoned adjustment/reversal records and a privileged actor.

HMO: draft invoice → posted → submitted claim → settlement

Resolve coverage and effective tariff at the service date; validate preauthorization where required. Posting freezes invoice amounts and payer shares. Claims reference the invoice; manual insurer submission is supported in initial scope. Track submitted, approved, partially_approved, rejected and settled states with event history. Resubmissions retain the same claim identity.

Reliable side effects

Commit critical records and the database-backed job/outbox entry together when delivery must be guaranteed. Workers run after commit, retry safely and deduplicate notifications/exports. Clinical writes must not depend on an external email or insurer service being available.

Audit every transition with actor and time. External network calls and long report generation stay outside row-locking transactions. Return actionable errors; never convert a failed save into a success notification.

17 / API contract & examples

Proposed contract; base path /api/v1. Authentication handshake uses /login and /logout.

Endpoints	Capability
GET /me; GET/POST /users	Current identity; admin-controlled staff management.
GET/POST /patients; GET/PATCH /patients/{id}	Patient search, registration, detail and versioned updates.
POST /patients/{id}/encounters; POST /appointments	Open shared encounter; create appointment.
POST /encounters/{id}/clinical-entries; /vitals	Document care; sign via /clinical-entries/{id}/sign.
POST /admissions; GET /beds?available=1	Admission creation; current bed availability.
POST /admissions/{id}/bed-allocations	Initial bed placement or transfer with locking.
POST /admissions/{id}/authorize-discharge; /discharge	Separate clinical authorization and completion.
POST /encounters/{id}/prescriptions; /investigation-orders	Create encounter-linked prescriptions and orders.
POST /prescriptions/{id}/sign; POST /dispenses	Authorize prescription; atomic partial/full dispensing.
POST /investigation-items/{id}/results; POST /results/{id}/release	Versioned result entry and authorized release.
GET /stock-balances; POST /purchases/{id}/post	Inventory view and atomic stock receipt.
POST /invoices/{id}/post; POST /claims; GET /reports/{type}	Finalize charges, create claim and retrieve authorized reports.

Request and response conventions

JSON uses snake_case. Patient create body: {first_name, last_name, sex, date_of_birth, phone, emergency_contact:{name, relationship, phone}}. Response: {data:{id:1042, mrn:"EC-001042", version:1}, message:"Patient created"}. MRN format is illustrative; generation is server-owned.

Lists: {data:[...], links:{...}, meta:{current_page, per_page, total}}. Support search, page, per_page, sort and allowlisted filters; cap per_page at 100. Use ISO-8601 timestamps with offsets; store instants in UTC and display the hospital's configured timezone.

Errors: 401 no session; 403 denied; 404 missing/not visible; 409 conflict or stale version; 419 expired CSRF/session; 422 {message, errors:{field:[...]}}; 429 throttled; 500 generic message + request_id. No stack traces or credentials in responses.

18 / Operations, backup & recovery

Proposed deployment baseline

Dedicated hospital server running a maintained Linux distribution, Nginx, PHP-FPM, a supported Laravel/PHP combination and MySQL 8.4 LTS. Pin actual versions at implementation; confirm extensions and runtime requirements against deployment documentation [S5]. If hospital IT requires Windows, adapt the service manager/web stack while keeping the same application boundaries.

Run Nginx, PHP-FPM, MySQL and queue workers as supervised services. Schedule Laravel tasks each minute. Set APP_ENV=production, APP_DEBUG=false; protect APP_KEY; serve only the public directory; configure SPA/API routing separately. Build frontend assets and install locked dependencies before delivery; do not use development servers in production.

Backup proposal: requires IT approval [S6]

Area	Proposed policy
Recovery targets	RPO ≤ 1 hour; RTO ≤ 4 hours. Targets, not achieved guarantees; validate on hospital hardware.
Backup contents	Consistent InnoDB database backup, required binary logs, matching private file snapshot, application release/version and protected encryption keys/configuration.
Schedule / retention	Nightly full backup; ship binary logs at least every 15 minutes; protect file changes within the same recovery window. Retain 30 daily and 12 monthly copies, subject to hospital retention policy.
Destination / ownership	Separate encrypted backup device plus rotated offline copy. Assign named IT backup operator and key custodian; alert on stale/failed backups.
Restore drill	Monthly isolated restore: recover database to selected time, restore consistent files, verify login and sample clinical records/attachments, measure actual recovery time.

Availability boundary

A single server is a single point of failure. UPS, disk monitoring and backups reduce risk; they do not provide live failover. A standby server is a later availability option. Keep an agreed paper downtime process and a controlled reconciliation procedure.

19 / Security & operational visibility

Control	Implementation requirement
Access	Deny by default; Laravel policies on each record/action [S3]. Active-account check each request; revoke sessions when staff access is removed. No shared staff accounts.
Browser sessions	Secure + HttpOnly session cookies; SameSite policy; CSRF protection; login throttling; explicit logout and inactivity timeout. Proposed timeout: 15 minutes, subject to clinical workflow review.
Sensitive data	Disk/backup encryption and approved access. APP_KEY and backup keys held separately with recoverable custody. No passwords, tokens or full clinical payloads in application logs.
Files	Private storage, allowlisted file types/sizes, sanitized names, content verification and malware scanning where available. Authorized streaming; short-lived exports and cleanup.
Audit	Record identity changes, patient lookups/exports, clinical signing, result release, stock adjustments and billing changes. Restrict audit writes and audit-reader access; archive to separately controlled storage.
Health / monitoring	Monitor uptime, disk space, MySQL health, slow queries, queue lag, failed jobs, backup freshness and certificate expiry. Use request IDs for error investigation.
Maintenance	Controlled LAN deployment window; tested migrations and rollback/restore plan. Stop incompatible writes, drain workers as required and smoke-test all roles after deployment.
Retention	Team must define clinical, finance, upload, audit and backup retention. Archive rather than hard-delete signed records; verify applicable hospital and jurisdictional requirements before production.

Normal users must not have operating-system or direct database access. Configure guest Wi-Fi isolation and firewall restrictions. These are proposed engineering controls, not a claim of legal certification or compliance.

20 / Frontend integration & delivery

Changes required in the existing React app

• Replace mock AuthContext and persisted bearer-token authentication with one session-aware auth provider. Wire a single QueryClientProvider and API client; bootstrap from /me and clear cached data on logout or 401.

• Use same-origin credentials and CSRF handling. The current /auth/login bearer-token hook and payload expectations must be updated to the proposed /login session contract; do not mix the two modes.

• Move patient/admission/stock state to API queries. Doctor and nurse screens must use the same patient and encounter IDs and invalidate relevant queries after successful writes.

• Map camelCase form fields to agreed snake_case API DTOs. Remove client-authoritative role choices. Render 422 field errors; show success only after a committed server response.

• Disable public preview routes in the production build, add route guards for navigation and retain backend policies as the enforcement layer. Remove bundled demo records from production data flows.

Phase	Deliverable and acceptance check
1. Foundation	Laravel + MySQL migrations, staff roles, sessions, policies, audit, health checks and tested backup/restore.
2. Patient registry	Registration/search/contact APIs; verify saved records survive restart and appear on a second workstation.
3. Clinical & admissions	Shared encounters, clinical forms, wards and discharge; test two concurrent allocations of one bed.
4. Diagnostics & pharmacy	Orders, versioned results, prescriptions, receipts and dispensing; test no negative stock or over-dispensing.
5. HMO & reporting	Coverage, tariffs, invoice snapshots, claims and exports; test duplicate-post retries and restricted report access.
6. Pilot & release	Representative staff UAT, realistic load test, permission matrix review, restore drill and downtime rehearsal.

21 / Review decisions & references

Decisions to confirm with the team

• Server OS, LAN/DNS/certificate ownership, concurrent staff count and expected attachment volume; size the server using pilot load tests.

• Hospital timezone, MRN numbering, patient duplicate/merge policy, who may sign/release clinical records, and approved role permissions.

• Whether discharge requires billing clearance; staff reset process without internet; patient coverage and insurer claim rules.

• Backup custody, retention, RPO/RTO, file capacity, downtime procedure and whether a standby server is needed.

Basis in the current repository

Reviewed: easycare2/src/routes/RoleBasedRoutes.tsx; src/context/AuthContext.tsx; src/store/useAuthStore.ts; src/api/client/apClient.ts; frontdesk Registration; nurse/doctor PatientManagementContext; HMO, pharmacy and diagnostics routes. backend/README.md has no Laravel implementation. Proposed tables/endpoints in this document are not existing migrations or deployed capabilities.

Primary technical references • accessed 4 November 2026

[S1] Laravel Sanctum: first-party SPA session authentication

https://laravel.com/docs/13.x/sanctum

[S2] Laravel database transactions

https://laravel.com/docs/13.x/database

[S3] Laravel authorization policies

https://laravel.com/docs/13.x/authorization

[S4] MySQL 8.4 InnoDB locking reads

https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

[S5] Laravel production deployment

https://laravel.com/docs/13.x/deployment

[S6] MySQL 8.4 backup and recovery

https://dev.mysql.com/doc/refman/8.4/en/backup-and-recovery.html

The architecture, entity model, operational targets and rollout plan are project-specific recommendations. References support the underlying framework mechanisms; they do not independently validate this hospital design.

22 / Full frontend review

Static inventory of all TS/TSX source files, plus fresh production, lint and compiler checks.

Evidence	Result
Source coverage	742 files; 483 interfaces/object-type declarations; 138 route declarations (129 distinct patterns).
Import graph	629 files reachable from main.tsx; this includes static/type imports and is not a count of tested browser screens.
Production build	PASS. Vite packages the app; build does not run the TypeScript checker.
TypeScript	FAIL: 154 diagnostics, including unresolved imports, invalid props and incompatible types.
ESLint	FAIL: 19 errors; 26 warnings.
Incomplete routes	13 explicit ComingSoonPage routes; 7 reachable literal links to unregistered dashboard paths.
Code duplication	54 byte-identical file groups; additional similar copies exist.
Backend status	backend/ contains only a README. PHP, MySQL and Docker are not installed in this environment.

Readiness verdict

The frontend cannot yet be confirmed production-ready. The database is a reviewed proposal with passing portable relationship tests, not an implemented or MySQL-certified backend. No frontend source was modified by this review.

All route declarations were assigned a proposed storage area. Field/type extraction covers every source file, including unused copies. Complex clinical forms still need approved validation schemas and clinician review; a JSON column alone is not proof of correctness.

23 / Frontend findings — identity & care

F01 / Critical: Clinical tables are global, not patient/encounter scoped

src/pages/doctor/patientProfile/hooks/useMedicalTable.ts:67

tables and pendingSectionEntries are keyed by section only. Require encounter-scoped storage/query keys; never attach one patient's rows to another.

F02 / Critical: Direct doctor profile URL can show the sample patient

src/pages/doctor/patientProfile/PatientProfile.tsx:101

Patient comes from location.state or MOCK_PATIENT; route :id is not resolved. Fetch by route ID and show 404/loading instead of fallback.

F03 / High: Two independent auth stores; production routes are public preview

src/routes/RoleBasedRoutes.tsx:100; src/context/AuthContext.tsx:40

Replace mock password/session storage, enforce Laravel policies, use one authenticated session and clear identity-scoped caches.

F04 / High: Patient registration reports success without saving

src/modules/frontdesk/pages/Registration/Registration.tsx:172

Submission logs formData and changes step. Backend transaction must save patient/contact/coverage; success follows committed response only.

F05 / High: Doctor ward route uses a nurse-only context

src/routes/RoleBasedRoutes.tsx:162; src/pages/nurse/patient-management/available-ward/components/WardBedsLog.tsx:47

DoctorLayout supplies a different provider. Code inspection indicates the nurse hook throws on this route. Replace with shared API state.

F06 / High: Several routed registration fragments receive no form state

src/modules/frontdesk/routes.tsx:40

Three step components require formData but are mounted directly without props. Route through the wizard or provide explicit loaders/state.

These findings are based on code inspection and compiler evidence. Browser reproduction and remediation validation remain required. Full locations and diagnostic files are in the review bundle.

24 / Frontend findings — integration & data

F07 / High: No operational API integration in clinical/stock/billing screens

src/api/client/apClient.ts:10

The API call sites found are auth hooks. The routed app uses mocks/local state; schema coverage alone cannot make these saves persist.

F08 / High: Export authorization uses a fixed demonstration OTP

src/constant/ExportOtpModal.tsx:9

1234 is a demo gate, not authorization. Protect export generation/download on the server; use real session reauthentication if required.

F09 / Medium: Navigation includes unregistered dashboard destinations

src/modules/frontdesk/pages/manage-card/ManageCard.tsx:50

Seven reachable literal navigation occurrences target /frontdesk/dashboard or /dashboard; wildcard silently returns to preview. Use agreed module home paths.

F10 / Medium: HMO registration loses tariff upload and organization detail

src/modules/hmo/pages/hmo-management/registration/components/HmoRegistrationForm.tsx:53

Only filename is retained; organizationName is not sent and contact person is derived from a phone field. Persist file_id and distinct provider/contact data.

F11 / Medium: Patient life status is mislabeled as insurance eligibility

src/modules/frontdesk/pages/Registration/PatientInsuranceReg.tsx:14

Alive/deceased options must map to patients.life_status, separately from coverage verification and staff access status.

F12 / Medium: Offline server deployment still pulls a Google font

src/index.css:1

Bundle the font and all required assets locally; verify hospital workflows with internet access disabled.

These findings are based on code inspection and compiler evidence. Browser reproduction and remediation validation remain required. Full locations and diagnostic files are in the review bundle.

25 / Frontend-to-database coverage

Storage coverage is proposed; the operational APIs are not implemented.

Workflow / screen family	Proposed tables
Preview / fallback	No persistent domain data
Staff identity / account	users, staff_profiles, staff_contacts, roles, permissions, user_roles, role_permissions, files
Patient registration / edit	patients, patient_categories, patient_contacts, patient_coverages, insurance_plans, hmo_providers, files, patient_identifiers
HMO organization registration	hmo_providers, insurance_plans, tariffs, tariff_imports, files, services
Card conversion / merge	patients, patient_identifiers, patient_category_history, patient_merges, patient_categories, audit_events
Patient access restrictions	patients, patient_access_events, users, audit_events
Patient registration log	patients, patient_categories, patient_contacts, users
Appointments	appointments, patients, users, doctor_assignments
Clinician assignment / queue	doctor_assignments, encounters, users
Previous patient records	patients, encounters, clinical_entries, documents, files, encounter_diagnoses, result_versions
Flags	patient_flags, patients, users, audit_events
Available wards / beds	wards, beds, admissions, bed_allocations, current_bed_occupancies
Admission / discharge	admissions, encounters, wards, beds, bed_allocations, current_bed_occupancies, users
Maternal, family-planning and immunization reports	care_programs, care_program_visits, clinical_entries, clinical_templates, encounters, appointments, deliveries, births, vaccinations, patient_relationships
Reports / nursing notes	clinical_entries, clinical_templates, encounters, users
Medication requests / requisitions	requisitions, requisition_items, departments, stores, drugs, encounters, stock_adjustments, stock_adjustment_items

Every one of the 138 route declarations is mapped individually in frontend-route-coverage.json and Frontend_Database_Review.md, both supplied with this document. Patient profile tables span clinical and financial domains; presentation-only fields stay out of the schema.

26 / Frontend-to-database coverage

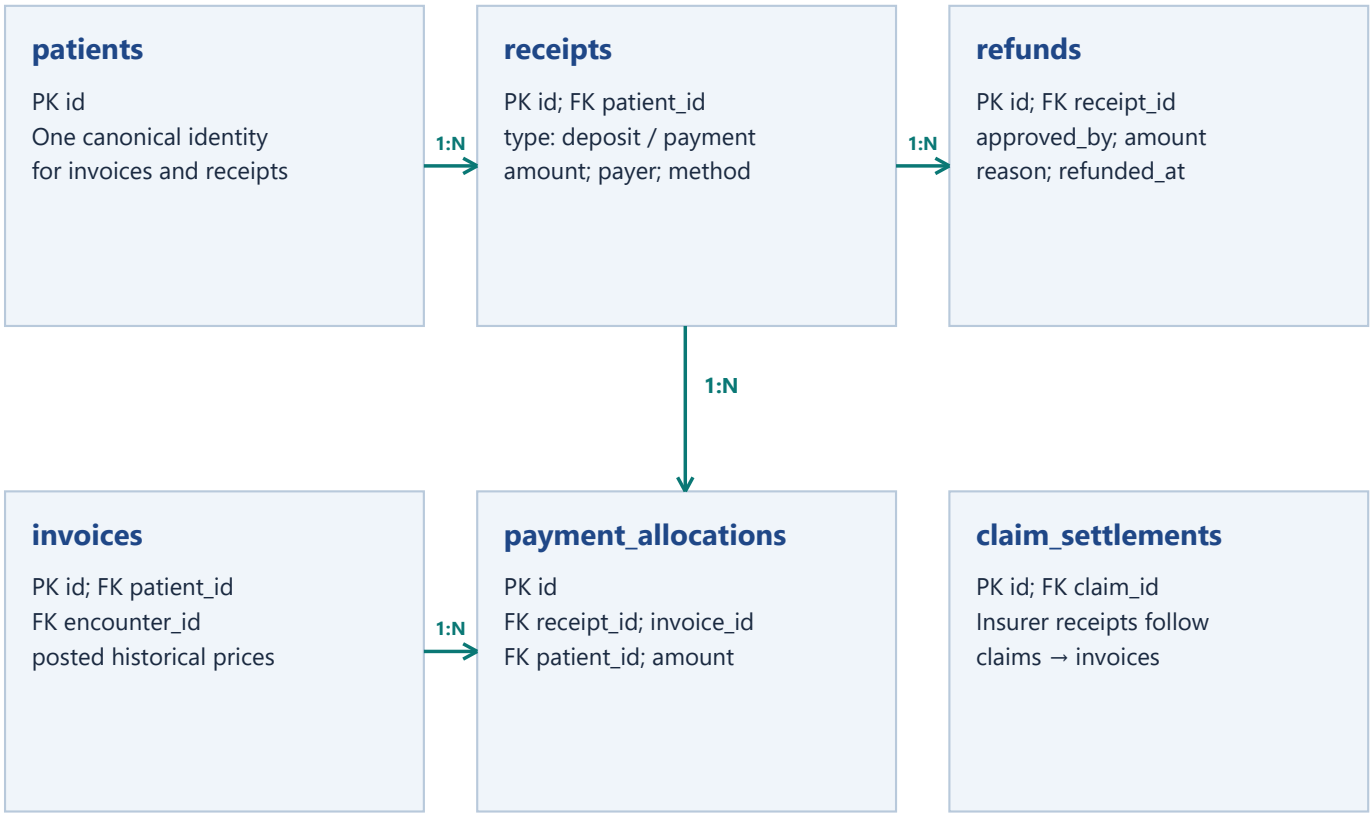
Storage coverage is proposed; the operational APIs are not implemented.

Workflow / screen family	Proposed tables
Prescriptions / dispensing reports	prescriptions, prescription_items, dispenses, dispense_items, medication_administrations, drugs, encounters
Reminders	reminders, notifications, users, patients
Notifications	notifications, users
Pharmacy stores / inventory	drugs, drug_prices, stores, store_drug_settings, drug_batches, stock_balances, stock_movements
Purchasing / suppliers	suppliers, purchases, purchase_items, drug_batches, stores, stock_movements, stock_balances
Batch tracking / damage / expiry	drug_batches, stock_balances, store_drug_settings, stock_adjustments, stock_adjustment_items, stock_movements
Medication return	stock_adjustments, stock_adjustment_items, stock_movements, dispense_items, refunds
Investigation catalog	investigation_tests, investigation_schemas, services
Investigation result entry	investigation_orders, investigation_items, investigation_tests, investigation_schemas, result_versions, documents, files
Investigation / visitation logs	encounters, patients, investigation_orders, investigation_items, result_versions
HMO accounts / bills	hmo_providers, patient_coverages, insurance_plans, tariffs, services, invoices, invoice_items, receipts, payment_allocations
Claims	claims, claim_items, claim_events, claim_settlements, invoices, invoice_items, preauthorizations
Insurance registration log	patient_coverages, patients, insurance_plans, hmo_providers
Patient clinical / financial profile	patients, encounters, clinical_templates, clinical_entries, vital_observations, encounter_diagnoses, performed_procedures, prescriptions, prescription_items, medication_administrations, vaccinations, investigation_items, result_versions, documents, files, invoices, invoice_items, receipts, payment_allocations, refunds, claims
Dashboards	patients, encounters, admissions, doctor_assignments, investigation_orders, stock_balances, invoices, notifications

Every one of the 138 route declarations is mapped individually in frontend-route-coverage.json and Frontend_Database_Review.md, both supplied with this document. Patient profile tables span clinical and financial domains; presentation-only fields stay out of the schema.

27 / Revised relationship: payments

A deposit exists before a bill. A payment can be allocated across more than one invoice.



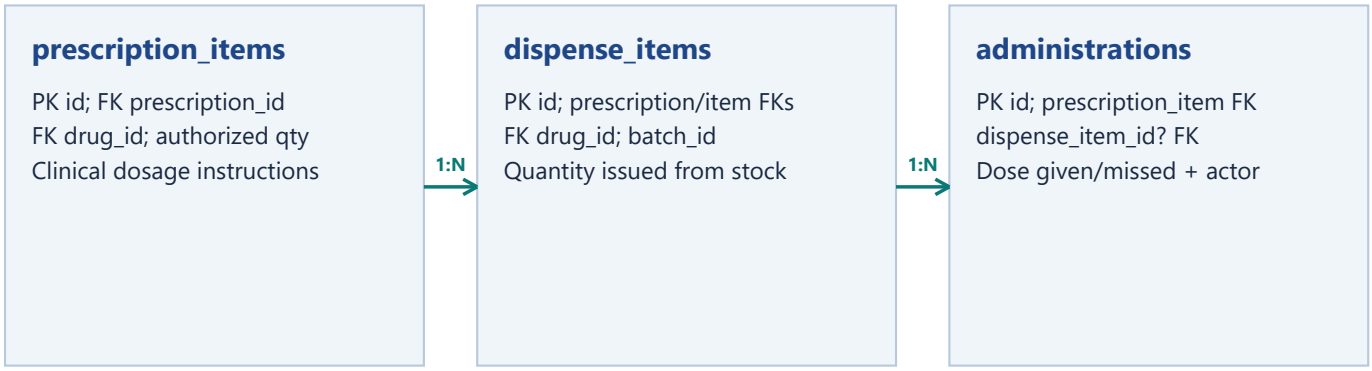
Invariant	Where it is enforced
One patient per allocation	Composite FKs require receipt.patient_id and invoice.patient_id to match the allocation patient.
No over-allocation or over-refund	Laravel locks receipt/invoice rows; allocated + refunded amounts must not exceed the receipt. Cross-row totals cannot be a simple CHECK.
No hidden balance copies	Outstanding/deposit credit is derived from invoice lines, allocations, refunds and insurer settlement rules.
Claim decisions	claim_items records approval/rejection amounts per invoice line. Services enforce same-invoice and service relationships.

28 / Revised relationships: identity & doses

Card conversion / merging

Relationship	Meaning
patients → patient_identifiers	1:N. Keep historical card IDs; register current MRN and old identifiers atomically.
patients → patient_category_history	1:N. Change commercial category with actor/reason; never replace the patient internal ID.
patients → patient_merges → patients	Source and target FKs. No self-merge; application prevents cycles and resolves canonical identity without losing original provenance.
patients → access_events / flags	Restrictions and flags are separate from staff role permissions and alive/deceased status.

Pharmacy issue / nursing administration



Composite keys prevent dispensing a line from a different prescription or using a batch of the wrong drug. Nursing administrations reference the correct prescription line and optional issue; they do not deduct pharmacy stock a second time.

Department/patient medication requests use requisitions + requisition_items, with request_scope defining which relationship is required. The diagnostics Make Request screen currently uses medication rows too; it must not be mistaken for a laboratory order form.

Pack sizes and buying/selling price snapshots, per-store reorder levels, damaged/expired adjustments and returns are explicitly represented. All posting services still need stock locks, expiry checks and cumulative quantity checks.

29 / What the schema tests prove

Executed evidence

85 table definitions and 201 foreign keys passed structural checks. All 36 portable relational scenarios passed. Engine: SQLite in-memory projection. Actual MySQL execution: NOT performed.

Validated locally	Still needs backend/runtime verification
Every FK has valid columns, matching types and an explicit unique/primary target.	Import generated DDL into MySQL 8.4; inspect engine, indexes, constraints, strict mode and decimal/date behavior.
MRN uniqueness, retained contact references and orphan/deletion prevention.	Canonical merge resolution, duplicate patient matching, identity authorization and all record-level policies.
Bed uniqueness and correct allocation/bed/admission linkage.	InnoDB row locks, simultaneous admissions, deadlock retries and release/transfer state transitions.
Negative stock, duplicate movement sources and cross-prescription/wrong-drug links are rejected.	No over-dispensing/over-return, issuing-store consistency, expiry/FEFO rules and ledger reconciliation.
Cross-patient invoices/documents/payments rejected; deposits can precede invoices.	Receipt allocation/refund limits, payer currencies, rounding, tariff overlaps and claim-line eligibility.
Result versions, correct test schema, verifier requirement and multiple births supported.	Approved JSON schemas, clinician semantics, immutable amendments and complete report projections.

The supplied verify_mysql.py imports the real DDL into a randomly named disposable database and runs the relational suite plus two NOWAIT row-lock tests. It has been syntax-checked only here. Passing it will not substitute for Laravel service tests or staff UAT.

30 / Implementation gates & handoff

Gate	Required acceptance evidence
1. Approve schema semantics	Agree life-status vs cover eligibility, category vs IPD/OPD, merge rules, patient restrictions, dispensing/administering, deposit allocation and per-line insurer decisions.
2. Validate real MySQL	Run verify_mysql.py on disposable MySQL 8.4. Confirm DDL import, checks, FK targets, dates/decimals and locks. Review the full dictionary before writing Laravel migrations.
3. Implement Laravel services	Session auth, policies, transaction services, immutable signing/amendment, idempotency, correct query scopes and server-side totals. Framework sessions/jobs tables come from Laravel migrations.
4. Fix frontend blockers	Remove mock patient fallback/global clinical state, unify auth, wire all saves, fix wrong providers and dead links; make lint/type checks pass; replace stubs before enabling them.
5. Exercise real workflows	Two staff browsers: register/reload; cross-module lookup; sign/order/release; allocate same bed; partially dispense same prescription; deposit/allocate/refund; discharge and re-admit.
6. Operational sign-off	Internet-disconnected LAN test, role review, restore drill, clinical UAT, realistic load test and controlled release/downtime plan.

New API areas identified by the full audit

Add versioned commands/resources for patient identifiers/category conversion/merges/access events; doctor assignments; nursing administrations; department/patient requisitions; tariff imports; receipts/allocations/refunds; stock adjustments/transfers; care programs/deliveries/births/vaccinations. Exact DTOs and privileges must be agreed before implementation.

Files for the backend team

schema.mysql.sql; Database_Dictionary_v1.1.md; Frontend_Database_Review.md; frontend-route-coverage.json; clinical-form-candidates.json; review-summary.json; schema test evidence and runners. The PDF embeds the SQL, dictionary, full review, route coverage and summary. The handoff ZIP includes all supporting sources.

No database was deployed and no frontend source changed. This review improves the proposal and exposes blockers; it does not certify an unimplemented system as perfect.